# 最近最少使用（LRU, Least Recently Used）

## 算法原理

LRU 认为**最近使用的数据在未来仍可能被访问**，因此它会淘汰**最久未被使用的数据**。

## 实现方式

1. **使用双向链表维护数据**，新插入的数据放在链表头部。
2. **每次访问数据**，将该数据移到链表头部，表示它是最近使用的数据。
3. **当缓存满时**，淘汰链表尾部的数据（即最近最少使用的数据）。
4. **使用哈希表存储 key 和链表节点的映射**，提高访问效率。

## 优缺点

### 优点

- **充分利用数据的时间局部性**，能较好适应大多数缓存场景。
- **实现相对简单，查询效率高（O(1)）**。

### 缺点

- **在批量操作场景下，可能导致缓存污染**，如周期性批量任务可能清空缓存，导致缓存命中率下降。

## KamaCache 的实现

### 数据结构设计

代码块

```go
type lruCache struct {
    mu              sync.RWMutex                 // 读写锁，保证并发安全
    list            *list.List                   // 双向链表，用于维护 LRU 顺序
    items           map[string]*list.Element // 键到链表节点的映射
    expires         map[string]time.Time     // 过期时间映射
    maxBytes        int64                        // 最大允许字节数
    usedBytes       int64                        // 当前使用的字节数
    onEvicted       func(key string, value Value) // 淘汰回调
    cleanupInterval time.Duration                // 清理间隔
    cleanupTicker   *time.Ticker                 // 定时器
    closeCh         chan struct{}                // 关闭通道
}
```

---

**关键设计点:**

1. **双向链表 + 哈希表**：结合 Go 标准库的 `container/list` 和内置 `map` 实现 O(1) 的查找和更新操作
2. **读写锁**：区分读写操作使用 `sync.RWMutex`，提高并发性能
3. **过期时间映射**：独立的 `expires` 映射，用于快速判断和管理项目过期
4. **内存追踪**：通过 `maxBytes` 和 `usedBytes` 限制和追踪内存使用
5. **资源回收**：通过 `onEvicted` 回调在删除缓存项时执行自定义逻辑
6. **自动清理**：使用 goroutine 和定时器实现后台自动清理

### 获取缓存项（`Get` 方法获取给定键的缓存项）

代码块

```go
func (c *lruCache) Get(key string) (Value, bool) {
    c.mu.RLock()
    elem, ok := c.items[key]
    if !ok {
        c.mu.RUnlock()
        return nil, false
    }

    // 检查是否过期
    if expTime, hasExp := c.expires[key]; hasExp && time.Now().After(expTime) {
        c.mu.RUnlock()
        // 异步删除过期项，避免在读锁内操作
        go c.Delete(key)
        return nil, false
    }

    // 获取值并释放读锁
    entry := elem.Value.(*lruEntry)
    value := entry.value
    c.mu.RUnlock()

    // 更新 LRU 位置需要写锁
    c.mu.Lock()
    // 再次检查元素是否仍然存在
    if _, ok := c.items[key]; ok {
        c.list.MoveToBack(elem)
    }
    c.mu.Unlock()

    return value, true
}
```

**实现要点：**

1. 先使用读锁检查项目是否存在和过期
2. 如果过期，异步删除该项（避免在读锁中修改数据）
3. 使用写锁更新 LRU 顺序（将访问的项移到链表尾部）
4. 使用二段锁定策略，减少锁的持有时间

### 设置缓存项

代码块

```go
func (c *lruCache) Set(key string, value Value) error {
    return c.SetWithExpiration(key, value, 0)
}

func (c *lruCache) SetWithExpiration(key string, value Value, expiration time.Duration) error {
    if value == nil {
        c.Delete(key)
        return nil
    }

    c.mu.Lock()
    defer c.mu.Unlock()

    // 计算过期时间
    var expTime time.Time
    if expiration > 0 {
        expTime = time.Now().Add(expiration)
        c.expires[key] = expTime
    } else {
        delete(c.expires, key)
    }

    // 如果键已存在，更新值
    if elem, ok := c.items[key]; ok {
        oldEntry := elem.Value.(*lruEntry)
        c.usedBytes += int64(value.Len() - oldEntry.value.Len())
        oldEntry.value = value
        c.list.MoveToBack(elem)
        return nil
    }

    // 添加新项
    entry := &lruEntry{key: key, value: value}
    elem := c.list.PushBack(entry)
    c.items[key] = elem
    c.usedBytes += int64(len(key) + value.Len())

    // 检查是否需要淘汰旧项
    c.evict()

    return nil
}
```

实现要点：

1. 支持可选的过期时间设置
2. 对于已存在的项，更新值并调整内存使用计数
3. 添加新项时，更新映射和链表
4. 每次添加后触发 `evict` 检查是否需要淘汰

### 淘汰策略

代码块

```go
func (c *lruCache) removeElement(elem *list.Element) {
    entry := elem.Value.(*lruEntry)
    c.list.Remove(elem)
```

```go
    delete(c.items, entry.key)
    delete(c.expires, entry.key)
    c.usedBytes -= int64(len(entry.key) + entry.value.Len())

    if c.onEvicted != nil {
        c.onEvicted(entry.key, entry.value)
    }
}

func (c *lruCache) evict() {
    // 先清理过期项
    now := time.Now()
    for key, expTime := range c.expires {
        if now.After(expTime) {
            if elem, ok := c.items[key]; ok {
                c.removeElement(elem)
            }
        }
    }

    // 再根据内存限制清理最久未使用的项
    for c.maxBytes > 0 && c.usedBytes > c.maxBytes && c.list.Len() > 0 {
        elem := c.list.Front() // 获取最久未使用的项（链表头部）
        if elem != nil {
            c.removeElement(elem)
        }
    }
}
```

淘汰策略分两步：

1. 清理所有已过期的项
2. 如果设置了内存限制，且当前使用的内存超过限制，从链表头部开始淘汰（最久未使用的项）

## LRU-K（Least Recently Used K）

### 算法原理

LRU-K 通过**维护访问历史**，只有当数据被访问**K 次**后，才将其放入缓存，以减少缓存污染。

### 实现方式

1. **所有数据首次访问时，进入历史访问队列**。
2. **如果数据访问次数达到 K 次**，才放入 LRU 缓存队列。
3. **当缓存满时**，淘汰“倒数第 K 次访问距离当前时间最长的数据”。

### 优缺点

#### 优点

- **减少缓存污染**，更适用于数据访问模式复杂的场景。

#### 缺点

- **需要维护额外的历史访问队列，占用更多内存。**

## LRU-2（Least Recently Used 2）

### 算法原理

LRU-2 是 LRU-K 算法的特例（K=2），即只有当某个数据被访问至少两次后，才可能被缓存。相比传统 LRU 更能抵抗缓存污染，适用于访问模式中存在临时热点的情况。

### 实现方式

1. 首次访问的数据被记录在历史访问队列（非缓存队列）中。
2. 当同一数据被第二次访问时，将其移入实际的 LRU 缓存队列。
3. LRU 缓存队列按最近访问顺序维护，最久未使用的数据靠前。
4. 缓存满时，淘汰 LRU 缓存队列中最久未访问的数据。

### 优缺点

#### 优点

- 能有效过滤一次性访问数据，减少缓存污染。
- 相比 LRU-K 更易实现，性能开销较小。

#### 缺点

- 与 LRU 相比，仍需维护一个额外的历史访问队列。
- 如果访问频率极低，则可能长时间无法进入缓存，影响命中率。

## KamaCache 的实现

### 核心设计理念

- **两级LRU策略**：解决传统LRU的"缓存污染"问题
- **分桶并发**：通过水平分片减少锁竞争
- **内存预分配**：避免运行时内存分配，减少GC压力
- **索引化双向链表**：使用数组索引代替指针，提升缓存局部性
- **自定义时钟**：减少系统调用，提升时间获取性能
- **主动+被动过期**：确保过期数据及时清理

### 节点结构（node）

代码块

```
type node struct {
    k        string // 键
    v        Value  // 值
    expireAt int64  // 过期时间戳，expireAt = 0 表示已删除
}
```

- **最小化内存占用**：只包含必要的三个字段
- **过期标记复用**：使用 `expireAt = 0` 表示节点已删除，避免额外的标记字段

- **类型抽象**：Value 是接口类型，支持存储任意类型的值

### 缓存结构

代码块

```
type cache struct {
        dlnk [][2]uint16       // 双向链表，0 表示前驱，1 表示后继
        m    []node             // 预分配内存存储节点
        hmap map[string]uint16 // 键到节点索引的映射
        last uint16             // 最后一个节点元素的索引
}
```

1. **索引化双向链表** ( `dlnk` ):
   - 不使用传统指针链表，而是用索引数组
   - `dlnk[i][0]` 存储节点i的前驱索引
   - `dlnk[i][1]` 存储节点i的后继索引
   - `dlnk[0]` 作为哨兵节点， `dlnk[0][0]` 存储尾节点索引， `dlnk[0][1]` 存储头节点索引
2. **预分配内存池** ( `m` ):
   - 启动时预分配固定大小的节点数组
   - 避免运行时动态分配内存
   - 通过索引复用节点，减少GC压力
3. **快速查找映射** ( `hmap` ):
   - 从键直接映射到节点索引
   - 实现O(1)的查找复杂度
4. **内存管理** ( `last` ):
   - 跟踪已分配的节点数量
   - 用于判断是否需要驱逐老数据

**内存布局示例：**

代码块

```
dlnk数组布局:
[0]: [尾索引, 头索引]  // 哨兵节点
[1]: [前驱, 后继]     // 节点1的链表关系
[2]: [前驱, 后继]     // 节点2的链表关系
...

m数组布局:
[0]: {key1, value1, expireAt1}  // 对应dlnk[1]
[1]: {key2, value2, expireAt2}  // 对应dlnk[2]
...
```

### 主缓存结构

代码块

```go
type lru2Store struct {
        locks       []sync.Mutex                    // 每个桶的独立锁
        caches      [][2]*cache                     // 每个桶包含两级缓存
        onEvicted   func(key string, value Value)   // 驱逐回调函数
        cleanupTick *time.Ticker                   // 定期清理定时器
        mask        int32                           // 用于哈希取模的掩码
}
```

1. **分桶并发控制**（`locks` + `caches`）：
   - 将数据分散到多个桶中，每个桶独立加锁
   - 减少锁竞争，提升并发性能
   - 桶数量为2的幂，便于使用位运算快速定位
2. **两级缓存架构**（`caches[][2]*cache`）：
   - `caches[i][0]`：第i个桶的一级缓存（频次过滤器）
   - `caches[i][1]`：第i个桶的二级缓存（热点数据）
   - 新数据进入一级缓存，二次访问才进入二级缓存
3. **事件通知机制**（`onEvicted`）：
   - 当数据被驱逐时触发回调
   - 支持应用层进行额外的清理操作

## 最少使用频率（LFU, Least Frequently Used）

### 算法原理

LFU 通过**统计数据的访问次数**来决定淘汰策略，**使用次数最少的数据将被淘汰**。

### 实现方式

1. **新数据插入缓存时**，设置其访问计数为 1，并放入队列。
2. **每次访问缓存中的数据**，该数据的访问计数加 1，并重新调整队列排序，使得访问次数较少的数据靠后。
3. **当缓存满时**，淘汰访问次数最少的数据（即队列末尾数据）。
4. **为了优化查询速度**，可以使用**哈希表存储数据**，结合**优先队列（heap）或平衡二叉搜索树**进行访问频率排序。

### 优缺点

#### 优点

- **对高频访问的数据有较好优化**，能提升热点数据的命中率。
- **适用于访问频率稳定的场景**，如 AI 训练数据缓存、热点文章推荐系统等。

#### 缺点

- **实现复杂度较高**，需要额外的存储来维护访问频率。
- **无法应对短期热点数据**，如果某个数据短时间内访问次数较多，但随后不再访问，可能会导致缓存污染（Cold Start 问题）。